UNIVERSIDAD POLITÉCNICA
BICENTENARIO

Práctica 7: Animatrónico.

Presenta:

Rubén Antonio Arredondo Villegas

Edgar Ivan Gutierrez Gutierrez

Roberto Leonel Mendoza Nuñez

Ailton Alejandro Trujillo López

Matricula:

24033101

24032957

24033370

24033482

Asignatura:

Sistemas embebidos

Profesor:

Hugo Antonio Méndez Guzmán

Silao, Gto. A 06 de noviembre del 2026

Contenido

Introducción 2

Marco Teórico 3

Desarrollo 4

Resultados 9

Observaciones y Conclusiones 11

Referencias 12

Lista de Figuras

Ilustración 1. Circuito del animatrónico 5
Ilustración 2 Molde de la máscara del animatrónico 10
Ilustración 3 Circuito todo ensamblado dentro de la mascara 10
Ilustración 4 Resultado final 11

Introducción

Los sistemas de automatización interactivos y los desarrollos animatrónicos requieren que un microcontrolador pueda tomar decisiones y coordinar múltiples actuadores a partir de una condición de entrada. Una de las aplicaciones más comunes en la robótica y el entretenimiento es la creación de sistemas reactivos que simulan reacciones ante la presencia de un usuario.
En esta práctica se realizó un sistema animatrónico utilizando una ESP32, un sensor PIR HC-SR501, un servomotor, un módulo reproductor MP3 y dos LEDs. El objetivo fue automatizar una respuesta multisensorial al detectar movimiento, haciendo que el sistema activara de manera sincronizada el movimiento de la boca, efectos de sonido e iluminación. De esta manera, se pudo observar cómo funciona la integración de sensores y diversos actuadores mediante sistemas embebidos y programación para lograr una interacción autónoma y dinámica.

Marco Teórico

El desarrollo de sistemas animatrónicos interactivos y prototipos automatizados requiere la integración de diversas áreas de la ingeniería, tales como la electrónica digital, los sistemas embebidos y el control de actuadores. Para lograr que un sistema responda de manera autónoma ante la presencia de un usuario, es indispensable comprender los fundamentos teóricos de cada uno de los componentes de hardware que participan en el circuito.

Los sistemas embebidos son dispositivos diseñados para realizar funciones dedicadas con altos niveles de eficiencia en tiempo real. El microcontrolador ESP32 destaca por su arquitectura de doble núcleo, conectividad inalámbrica y una amplia gama de periféricos de entrada y salida [1]. En aplicaciones de control, sus pines de propósito general (GPIO) se configuran digitalmente para interactuar con sensores y actuadores, procesando la lógica del sistema mediante programas estructurados en entornos de desarrollo como Arduino IDE [2].

Para la detección de movimiento en sistemas reactivos se emplean sensores infrarrojos pasivos como el HC-SR501, los cuales identifican cambios en la radiación térmica emitida por cuerpos en movimiento dentro de su campo de visión [3]. Este dispositivo utiliza una lente Fresnel dividida en sectores para enfocar la radiación hacia un elemento piroeléctrico dual, generando una variación de voltaje que el circuito interno transforma en una señal digital de disparo hacia el microcontrolador [3].

Al recibir dicha señal, el controlador activa la respuesta física del animatrónico a través de actuadores electromecánicos como el servomotor SG90. Este tipo de servomotor funciona como un sistema de bucle cerrado cuya posición angular está determinada por la técnica de Modulación por Ancho de Pulso (PWM), en la cual la duración de cada pulso eléctrico

enviado desde la ESP32 fija con precisión el ángulo de rotación del eje para simular movimientos biológicos como la apertura y cierre de la boca [4].

De forma simultánea, la integración de elementos de audio enriquece la experiencia interactiva mediante el uso de módulos reproductores independientes como el DFPlayer Mini. Este dispositivo cuenta con un chip decodificador de audio que gestiona archivos en formato MP3 o WAV guardados en una tarjeta de memoria microSD, comunicándose con la ESP32 a través de comunicación serial (UART) para reproducir efectos de sonido o voz por medio de un altavoz acoplado [5]. Finalmente, los diodos emisores de luz (LEDs) actúan como elementos de iluminación complementarios, requiriendo el uso de resistencias limitadoras conectadas en serie para regular el paso de la corriente eléctrica según la Ley de Ohm y prevenir daños en los semiconductores [6].

Material requerido

- 1x ESP32
- 1x Sensor PIR HC-SR501
- 1x Servomotor SG90
- 2x LEDs rojos o de otro color
- 2x Resistencias de $220 - 330\ \Omega$
- 1x Módulo reproductor MP3 (DFPlayer Mini)
- 1x Tarjeta microSD compatible
- 1x Altavoz compatible con el reproductor
- Protoboard y cables Dupont
- 1x Fuente regulada adecuada

Desarrollo

Al comenzar la práctica, primero se revisaron todos los componentes para verificar que funcionaran de manera adecuada. Una de las primeras actividades fue comprobar el estado del sensor PIR HC-SR501 y del servomotor, asegurando que este último respondiera correctamente a las señales de control antes de integrarlo en la estructura de la mandíbula. Asimismo, se probó la tarjeta microSD y el módulo reproductor MP3 para confirmar que los archivos de audio se reprodujeran sin inconvenientes a través del altavoz.

Una vez comprobado el funcionamiento individual de los elementos, se procedió a realizar las conexiones eléctricas sobre el protoboard. El sensor PIR se conectó a la ESP32 para recibir la alimentación de voltaje y enlazar su pin de salida digital, encargado de enviar la señal cuando se detectara movimiento en el entorno. Posteriormente, se montaron los dos LEDs para simular los ojos del sistema, conectándolos en serie con sus respectivas

resistencias de protección para evitar daños por sobrecorriente al ser activados por los pines de salida digital del microcontrolador.

En cuanto al sistema de audio y movimiento, el servomotor se fijó mecánicamente para accionar directamente la mandíbula del sistema, conectando su cable de señal a un pin PWM de la ESP32 para regular su apertura y cierre de forma sincronizada. Por su parte, el módulo reproductor MP3 se interconectó mediante comunicación serial (pines TX y RX) con la tarjeta de desarrollo, vinculando el altavoz a su salida de audio para permitir la emisión de los sonidos coordinados.

La alimentación general del sistema se distribuyó conectando la fuente regulada y los pines de la ESP32, asegurando que tanto los actuadores como los módulos multimedia dispusieran de la energía estable necesaria para operar de manera simultánea al activarse el sensor de presencia

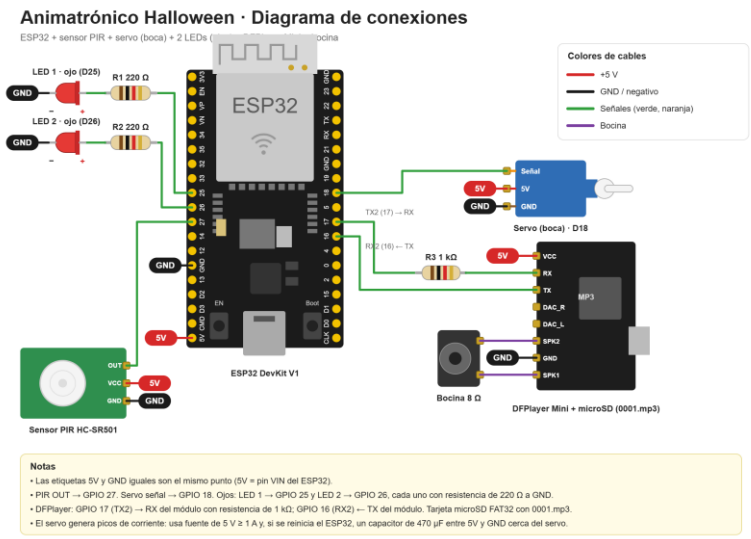


Ilustración 1. Circuito del animatrónico

El diagrama mostrado anteriormente representa la conexión completa de todos los componentes que integran el sistema animatrónico de Halloween. En él se puede observar la interacción entre el microcontrolador ESP32, el sensor PIR HC-SR501, el servomotor de la boca, los dos LEDs que simulan los ojos, el módulo reproductor MP3 (DFPlayer Mini) con su respectiva tarjeta microSD y la bocina.

El ESP32 funciona como la unidad principal de control, ya que recibe la información de presencia proporcionada por el sensor PIR y procesa los datos para tomar decisiones de

acuerdo con la programación establecida. Dependiendo de si se detecta movimiento, el microcontrolador envía señales simultáneas para activar los actuadores y dispositivos multimedia.

Específicamente, el ESP32 comanda el encendido de los dos LEDs indicadores a través de los pines digitales (protegidos con resistencias de $220\ \Omega$), envía una señal PWM por el pin D18 para controlar el movimiento del servomotor que acciona la mandíbula, y se comunica mediante protocolo serial (RX/TX) con el módulo reproductor MP3 para reproducir el archivo de audio pregrabado en la tarjeta microSD y emitirlo a través de la bocina.

A continuación, se muestra el código empleado en el IDLE de arduino.

```
#include <ESP32Servo.h>
#include <DFRobotDFPlayerMini.h>

// ---------- Pines ----------
const int PIN_PIR   = 27;
const int PIN_SERVO = 18;
const int PIN_LED1  = 25;
const int PIN_LED2  = 26;
const int PIN_DF_RX = 16;   // recibe del TX del MP3
const int PIN_DF_TX = 17;   // envía al RX del MP3

// ---------- Ajustes ----------
const int  ANGULO_CERRADO = 0;
const int  ANGULO_ABIERTO = 50;
const int  VELOCIDAD_BOCA = 300;             // ms por movimiento
const unsigned long DURACION_SHOW     = 8000;  // ms que dura el show
const unsigned long PIR_LOW_ESTABLE   = 1000;  // ms con PIR en LOW para rearmar
const unsigned long ESPERA_MAX_PIR    = 15000; // máximo que espera a que el PIR baje
const unsigned long CALIBRACION_PIR   = 30000; // ms de calibración al encender
const int  VOLUMEN = 10;                  // 0 a 30
const int  PISTA   = 1;                   // 0001.mp3
const bool LEDS_SIEMPRE_ENCENDIDOS = false;    // true = ojos siempre prendidos

Servo servoBoca;
DFRobotDFPlayerMini mp3;
bool mp3Listo = false;
bool pirAnterior = false;

void ponerLeds(bool estado) {
  digitalWrite(PIN_LED1, estado);
  digitalWrite(PIN_LED2, estado);
}
```

```
void setup() {
  Serial.begin(115200);
  delay(500);
  Serial.println();
  Serial.println("=== Animatronico Halloween ===");

  pinMode(PIN_PIR, INPUT);
  pinMode(PIN_LED1, OUTPUT);
  pinMode(PIN_LED2, OUTPUT);
  ponerLeds(LEDS_SIEMPRE_ENCENDIDOS);

  servoBoca.setPeriodHertz(50);
  servoBoca.attach(PIN_SERVO, 500, 2400);
  servoBoca.write(ANGULO_CERRADO);

  Serial2.begin(9600, SERIAL_8N1, PIN_DF_RX, PIN_DF_TX);
  delay(1000);
  if (mp3.begin(Serial2)) {
    mp3Listo = true;
    mp3.volume(VOLUMEN);
    Serial.println("MP3 listo");
  } else {
    Serial.println("No se detecta el MP3: revisa conexiones y tarjeta SD");
  }

  // Calibración del PIR, mostrando el avance cada 5 s
  Serial.println("Calibrando sensor PIR (30 s)...");
  unsigned long inicioCal = millis();
  while (millis() - inicioCal < CALIBRACION_PIR) {
    delay(5000);
    Serial.print("  ... ");
    Serial.print((millis() - inicioCal) / 1000);
    Serial.println(" s");
  }

  // Esperar a que el PIR baje a LOW (máximo 10 s)
  unsigned long t0 = millis();
  while (digitalRead(PIN_PIR) == HIGH && millis() - t0 < 10000) {
    Serial.println("PIR en HIGH, esperando...");
    delay(500);
  }

  pirAnterior = digitalRead(PIN_PIR);
  Serial.println("Sistema listo");
}

void iniciarShow() {
  Serial.println("Movimiento detectado!");

  if (mp3Listo) mp3.play(PISTA);
  ponerLeds(true);                  // ojos fijos
```

```
  unsigned long inicio = millis();
  bool abierta = false;

  while (millis() - inicio < DURACION_SHOW) {
    abierta = !abierta;
    servoBoca.write(abierta ? ANGULO_ABIERTO : ANGULO_CERRADO);
    delay(VELOCIDAD_BOCA);
  }

  // Terminar
  servoBoca.write(ANGULO_CERRADO);
  ponerLeds(LEDS_SIEMPRE_ENCENDIDOS);
  if (mp3Listo) mp3.stop();

  // Esperar a que el PIR se mantenga en LOW (con límite de tiempo)
  Serial.println("Esperando a que el PIR se calme...");
  unsigned long tiempoLow = millis();
  unsigned long tMax = millis();
  while (millis() - tiempoLow < PIR_LOW_ESTABLE && millis() - tMax < ESPERA_MAX_PIR) {
    if (digitalRead(PIN_PIR) == HIGH) tiempoLow = millis();  // reinicia la cuenta
    delay(20);
  }

  if (digitalRead(PIN_PIR) == HIGH) {
    Serial.println("Aviso: el PIR sigue en HIGH (revisa potenciometro Tx y jumper)");
  }
  Serial.println("Listo para detectar de nuevo");
}

void loop() {
  bool pirActual = digitalRead(PIN_PIR);

  // Solo reacciona al pasar de LOW a HIGH (movimiento nuevo)
  if (pirActual && !pirAnterior) {
    iniciarShow();
    pirAnterior = digitalRead(PIN_PIR);   // estado real después del show
  } else {
    pirAnterior = pirActual;
  }

  delay(50);
}
```

Resultados

La instalación de todos los componentes se realizó correctamente y las conexiones en la protoboard fueron verificadas antes de energizar el circuito. Posteriormente, el código fue cargado con éxito en la ESP32 sin presentar errores de compilación ni de comunicación.

Durante las pruebas iniciales, el sistema funcionó de manera adecuada y totalmente funcional. Al encontrarse en estado de reposo, el sensor PIR HC-SR501 se mantuvo a la espera de movimiento; al momento de detectar la presencia de una persona en su campo de visión, el sensor envió la señal digital correspondiente a la ESP32.

De forma inmediata y coordinada, el microcontrolador activó la secuencia programada: los dos LEDs indicadores se encendieron correctamente, el servomotor ejecutó el movimiento mecánico de apertura y cierre de la mandíbula mediante señales PWM, y el módulo reproductor MP3 (DFPlayer Mini) reprodujo de manera clara y sincronizada el archivo de audio almacenado en la tarjeta microSD a través de la bocina.

De esta manera, se logró un sistema animatrónico completamente automatizado e interactivo mediante el uso de la ESP32, garantizando una respuesta eficiente y síncrona entre la detección de movimiento, la iluminación, el audio y la acción mecánica de acuerdo con los parámetros establecidos

A continuación, se mostrarán imágenes de los resultados junto con un video del funcionamiento:

Video del funcionamiento del animatrónico:

https://drive.google.com/file/d/1hRXZOAvRn26jJlOl5_RrIlcrcttAHm0P/view?usp=drive_link



Ilustración 2 Molde de la máscara del animatrónico

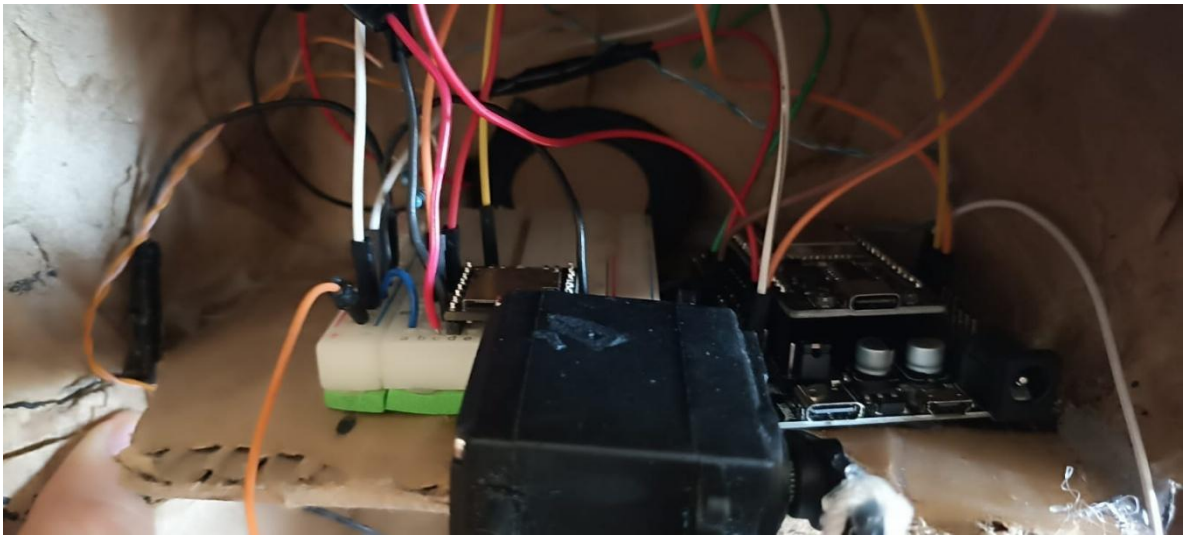


Ilustración 3 Circuito todo ensamblado dentro de la mascara



Ilustración 4 Resultado final

Observaciones y Conclusiones

En conclusión, la práctica se llevó a cabo con éxito, logrando implementar de manera correcta un sistema animatrónico interactivo controlado por un microcontrolador ESP32. A través de la integración del sensor PIR HC-SR501, fue posible detectar el movimiento en el entorno para desencadenar de forma automática y sincronizada una respuesta multisensorial.

Durante el desarrollo se comprobó la importancia de una correcta vinculación entre los componentes de hardware y la lógica de programación; el servomotor respondió adecuadamente para simular el movimiento de la mandíbula, los LEDs operaron como

indicadores visuales eficientes, y el módulo reproductor MP3 junto con la bocina emitieron de manera clara el audio pregrabado al activarse el sistema.

De esta manera, se cumplió con el objetivo principal de la práctica, evidenciando el funcionamiento de los sistemas embebidos en proyectos de automatización y robótica, y consolidando los conocimientos teóricos y prácticos sobre el manejo de entradas, salidas digitales, señales PWM y comunicación serial.

Referencias

[1] Espressif Systems. (2021). *ESP32 Series Datasheet: Version 3.5*. Espressif Systems.

[2] Monk, S. (2016). *Programming Arduino: Getting Started with Sketches* (2nd ed.). McGraw-Hill Education.

[3] Panditharatne, A., & Perera, S. (2019). *Design and Implementation of Motion-Activated Embedded Systems using PIR Sensors*. Journal of Embedded Electronics, 14(2), 45-52.

[4] Chapman, S. J. (2012). *Electric Machinery Fundamentals* (5th ed.). McGraw-Hill.

[5] DFRobot. (2018). *DFPlayer Mini Module Datasheet V2.0*. DFRobot Documentation.

[6] Boylestad, R. L., & Nashelsky, L. (2013). *Electronic Devices and Circuit Theory* (11th ed.). Pearson.